S.No: 2	Exp. Name: ***Merge Sort - Divide and Conquer***	Date:2026-08-04

Aim:

You are given an array of n integers, and your task is to sort this array in non-decreasing order using the Merge Sort algorithm, which is based on the Divide and Conquer strategy.

Implement Merge Sort by defining the following user-defined functions:

- **mergeSort()** - recursively divides the array into subarrays until each subarray has a single element.
- **merge()** - merges two sorted subarrays into a single sorted array.

Input Format:

- The first line contains a single integer n representing the number of elements in the array.
- The second line contains n space-separated integers representing the elements of the array.

Output Format:

- Print the sorted array as a single line of n space-separated integers.

Constraints:

- $1 \leq n \leq 10^6$
- $-10^9 \leq Array\ Elements \leq 10^9$

Source Code:

merge.c

```
#include <stdio.h>
#include <stdlib.h>

void merge(  int *arr, int l, int m, int r              ) {
    //write your code here..
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    // Create temporary arrays
    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    // Copy data to temporary arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        j[R] = arr[m + 1 + j];

    // Merge the temporary arrays back into arr[l..r]
    i = 0; // Initial index of first subarray
    j = 0; // Initial index of second subarray
    k = l; // Initial index of merged subarray
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
```

```
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    // Free allocated temporary memory
    free(L);
    free(R);



}

void mergeSort( int *arr, int l, int r               ) {
    //write your code here..
    if (l < r) {
        // Same as (l+r)/2, but avoids overflow for large l and h
        int m = l + (r - l) / 2;

        // Sort first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        // Merge the sorted halves
        merge(arr, l, m, r);
        }



}

int main() {
    int n;
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);
```

```
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
9 -5 7 -2 -1
-5 -2 -1 7 9

Test Case - 2
User Output
7
2 9 1 6 3 10 7
1 2 3 6 7 9 10

Test Case - 3
User Output
5
3 7 4 1 5
1 3 4 5 7